

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student & Hardware Metadata Field	Laboratory Session Specifications & Target Machine Declaration
Student Full Name	Serketay Nurdaulet
Student Identification Number	230103358
Date & Laboratory Session	CSS 314
Workstation / Laptop Make & Model	Macbook M2
Processor SKU / SoC Model	Apple M2
Core Topology (Physical P-cores / E-cores)	Physical Cores: <u>8</u> Logical SMT Threads: <u>8</u>
Cache Hierarchy Capacity	L1: <u>128</u> KB L2: <u>20</u> MB L3: <u> </u> MB
Host OS & Python Runtime	macOS Sonoma / Sequoia, CPython 3.11/3.12 (ARM64).

■ MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where $S(p)$ is the speedup ratio $T(1)/T(p)$, P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ EXECUTION PROTOCOL

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline $T(1)$ and the measured multi-core execution times across all 6 core allocations.

```

import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")

```

Worker Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency [S(p)/p * 100]
p = 1 (Baseline)	1.8258	1.00x	1.00x	100.0%
p = 2	0.9260	1.97x	2.00x	98.6%
p = 4	0.5576	3.27x	4.00x	81.9%
p = 8	0.5126	3.56x	8.00x	44.5%
p = 12	0.4930	3.70x	12.00x	30.9%
p = 16	0.5101	3.58x	16.00x	22.4%

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

Student Response & Architectural Justification:

The observed speedup curve reaches its primary inflection point at $p^* = 4$. Between $p = 1$ and $p = 4$, execution time drops from 1.8258s to 0.5576s, achieving a 3.27x speedup. This performance gain occurs because the macOS scheduler assigns these 4 worker processes directly to the Apple M2's physical Performance (P) cores (Avalanche microarchitecture @ 3.49 GHz). From $p = 4$ to $p = 8$, speedup flattens from 3.27x to 3.56x. This happens because additional workers spill over onto the 4 physical Efficiency (E) cores (Blizzard @ 2.42 GHz), which have lower clock frequencies and narrower execution pipelines. Beyond $p = 8$ ($p = 12$ and $p = 16$), performance stalls and degrades (3.70x at $p = 12$, falling to 3.58x at $p = 16$). The Apple M2 lacks SMT/Hyper-Threading and has exactly 8 physical cores. Running 12 or 16 workers causes severe CPU oversubscription, context-switching overhead, and IPC serialization in Python's multiprocessing.

[Q1.2] Using your empirical measurements for $T(1)$ and $T(4)$, calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

Student Response & Architectural Justification:

Using the measured baseline $T(1) = 1.8258\text{s}$ and four-core execution time $T(4) = 0.5576\text{s}$:

$$S(4) = T(1) / T(4) = 1.8258 / 0.5576 = 3.2744$$

$$S(4) = 1 / [(1 - P) + P / 4] = 3.2744$$

Algebraic derivation:

$$(1 - P) + 0.25 * P = 1 / 3.2744$$

$$1 - 0.75 * P = 0.3054$$

$$0.75 * P = 1 - 0.3054 = 0.6946$$

$$P = 0.6946 / 0.75 = 0.9261$$

The strictly parallel fraction of this workload is $P = 0.9261$ (92.61%).

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\text{max}} = \lim(p \rightarrow \infty) S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. (8 Points)

Student Response & Architectural Justification:

This limit proves that the 7.39% sequential fraction (comprising process spawn latency, memory setup, and IPC serialization) hard-caps the maximum speedup at 13.53x regardless of core count. Purchasing a 128-core server node for this specific workload would result in severe resource underutilization. Scaling from 8 cores to 128 cores would yield negligible performance gains beyond the 13.53x ceiling, leaving over 89% of the server's compute capacity idle.

$$S_{\text{max}} = \lim(p \rightarrow \infty) S(p) = 1 / (1 - P) = 1 / (1 - 0.9261) = 1 / 0.0739 = 13.53\text{x}$$

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements ($16 * 8 \text{ bytes} = 128 \text{ bytes}$, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```

import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")

```

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Penalty Multiplier
Adjacent Indices (False Sharing Active)	0.6863s	0.7007	0.6895	0.6895	— [Baseline]
Padded Indices (Cache-Line Aligned)	0.7244	0.6359	0.6428	0.6428	1.07x

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8 Points)

Student Response & Architectural Justification:

In 64-bit CPython, an integer reference/pointer inside a list occupies exactly 8 bytes (64 bits) of memory. With an index stride of 16 elements, the memory offset between elements accessed by adjacent threads is calculated as $16 \text{ elements} \times 8 \text{ bytes/element} = 128 \text{ bytes}$. Standard x86_64 CPU architectures utilize 64-byte L1 cache lines, while Apple Silicon (M-series) architectures feature 128-byte L1 cache lines. Because 128 bytes covers two full 64-byte cache lines (or one full 128-byte cache line on Apple M2), array elements separated by a stride of 16 are guaranteed to be placed on completely distinct cache lines. Mathematically, no two threads modifying adjacent indices in the padded array will ever access memory within the same 64-byte L1 cache line, fully eliminating hardware false sharing.

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). (9 Points)

Student Response & Architectural Justification:

Initially, both Core 0 and Core 1 load the shared cache line containing array elements into their respective L1 caches in the Shared (S) state. When Core 0 executes an increment on index 0, it requires exclusive write access and broadcasts a Bus Invalidate message across the interconnect. Core 0 updates its cache line state from Shared (S) to Modified (M), while forcing Core 1 copy of the cache line to transition from Shared (S) to Invalid (I). When Core 1 subsequently attempts to increment index 1, an L1 cache miss occurs due to its Invalid (I) state. Core 1 stalls its pipeline, issues a Read-For-Ownership (RFO) request, and forces Core 0 to write back its modified data. Core 1 receives the updated line, marks its state as Modified (M), and invalidates Core 0 copy (M -> I). This rapid bouncing of Bus Invalidate messages and state shifts (Shared -> Modified -> Invalid) causes severe cache line ping-ponging and CPU pipeline stalls.

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). (8 Points)

Student Response & Architectural Justification:

Two industry-standard mechanisms used to eliminate false sharing without manual array padding are:

C++11 Alignment Specifiers (alignas): Modern C++ provides the alignas specifier, such as alignas(64) or alignas(128). This explicitly forces the compiler to allocate structure fields or variables on memory addresses aligned to specific cache line boundaries. By ensuring each thread-local field occupies its own aligned cache line, false sharing is eliminated at compile time without manual padding code.

Java Virtual Machine Contended Annotation (@Contended): In Java (JDK 8+), the sun.misc.Contended annotation instructs the JIT compiler to automatically inject memory padding around specific fields or classes. The JVM dynamically detects the host hardware cache line size at runtime and inserts padding bytes around annotated fields, preventing false sharing while keeping code clean and platform-independent.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse-grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time
```

```
TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```

class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")

```

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead
Unsafe Accumulator (Race Condition)	2,000,000	2,000,000	0	0.1570	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	0.4747	3.02x

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read-modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

Student Response & Architectural Justification:

In this specific run, the actual empirical sum reached 2,000,000, resulting in 0 data corruption defect count. In CPython, simple thread execution loops sometimes complete within a single Global Interpreter Lock (GIL) time slice before the operating system or Python runtime triggers a thread preemption. However, the Python operation `val += 1` is non-atomic and inherently vulnerable to race conditions. At the ARM64 machine code level, it expands into three distinct instructions:

LOAD: Read memory address of `self.val` into CPU register R1.

ADD: Increment register value ($R1 = R1 + 1$).

STORE: Write register value R1 back to memory address of `self.val`.

If a thread switch occurs between LOAD and STORE, Thread 0 reads value N into register R1. Thread 1 immediately reads value N into register R2. Both threads increment their respective registers to N + 1 and write them back to memory. As a result, two separate increment operations produce a net gain of only 1, causing a lost update defect.

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x.

Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. (11 Points)

Student Response & Architectural Justification:

```
import threading
class LocklessCounter:
def init(self, num_threads=4):
self.buffers = [0] * num_threads

def work(self, thread_id, ops_per_thread):
local_acc = 0
for _ in range(ops_per_thread):
local_acc += 1
self.buffers[thread_id] = local_acc

def reduce(self):
return sum(self.buffers)
```

Measured execution time: 0.0465s

Speedup factor over LockedCounter: 10.21x (10.21x speedup easily surpasses the required 2.0x threshold).

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

Student Response & Architectural Justification:

Eliminating shared mutable state via thread-local buffers and a final reduction phase is fundamentally superior to optimizing lock primitives (such as spinlocks or read-write locks). Locks and spinlocks still force multiple CPU cores to contend over a single shared memory address holding the lock variable or counter. This ongoing contention causes constant cache line invalidation signals across interconnect buses, forces pipeline stalls, and triggers kernel context switches. By contrast, lockless partitioning isolates each worker thread to its own private memory buffer. Threads run at maximum hardware execution speeds without waiting for thread locks or invalidating L1 cache lines of other cores, reducing thread coordination costs to a single, trivial reduction step after threads complete.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple

cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```
import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()
```

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 1 Worker (Baseline)	0.8853	1.00x	0.6021	1.00x
w = 2 Workers	1.0207	1.15x	0.7573	1.26x

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 4 Workers	1.1286	1.27x	1.1925	1.98x

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

Student Response & Architectural Justification:

The compute-bound workload features high arithmetic intensity, executing floating-point math directly within CPU registers and ALU execution units with zero DRAM access. When scaling from 1 to 4 workers, each process runs on an independent physical Performance core. Because register math does not contend for shared resources, total wall-clock time increases only slightly from 0.8853s to 1.1286s (a 1.27x factor) despite doing 4x total work, demonstrating near-linear throughput scaling. By contrast, the memory-bound workload streams large arrays to and from DRAM with minimal arithmetic. Scaling to 4 workers causes concurrent memory requests to saturate the host system's memory controller and shared bus. This memory bus contention and queuing latency causes execution time to spike by 1.98x (from 0.6021s to 1.1925s), clearly illustrating DRAM bandwidth saturation under the Roofline Model.

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

Student Response & Architectural Justification:

the host machine is an Apple M2 MacBook Air equipped with Unified LPDDR5 SDRAM running at a memory clock speed of 6400 MT/s over a 128-bit wide memory bus interface.

Peak Bandwidth = (Bus Width in Bytes) * (Transfer Rate)

Peak Bandwidth = (128 bits / 8 bits/byte) * 6.4 G transfers/sec = 16 bytes * 6.4 G transfers/sec = 102.4 GB/s (rated at 100 GB/s).

Memory bus width and bandwidth limit multi-core scalability because all physical CPU cores share this single 100 GB/s unified memory channel. When multiple cores simultaneously stream large data arrays, their combined bandwidth demand exceeds the physical 100 GB/s limit. Once the memory bus is saturated, adding more worker cores cannot increase memory throughput; it only increases memory controller queuing delays and bus arbitration overhead.

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. (8 Points)

Student Response & Architectural Justification:

Optimizing the pipeline with CUDA GPU kernels would yield significantly greater speedup than CPU OpenMP pragmas. In a memory-bandwidth-saturated regime, system performance is strictly bounded by memory throughput rather than raw FLOPS compute capacity. CPU OpenMP pragmas scale execution across CPU cores that remain bottlenecked by the shared CPU memory bus (~100 GB/s on M2 or ~50-90 GB/s on standard x86 systems). By contrast, GPU architectures utilize dedicated high-bandwidth memory subsystems (HBM3 or GDDR6) providing bandwidth ranging from 900 GB/s to over 3 TB/s (or high-bandwidth unified memory on high-end chipsets up to 800 GB/s). Moving a memory-bound high-dimensional embedding vector pipeline to GPU kernels leverages this vastly superior memory bandwidth, bypassing CPU memory controller saturation and delivering orders of magnitude higher vector throughput.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Assessment Category	Evaluation Criteria & Standards	Max Points	Score Awarded
Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
☐ Task 3 Unsafe Corruption Verified ☐ Task 2 Cache Slowdown Verified ☐ Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

Operating System	Command to Retrieve CPU SKU & Core Topology	Command to Inspect Cache & Memory Bus
Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1d cachesize hw.l2 cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

```
import time
```

```
from multiprocessing import Pool
```

```
import threading
```

```
def cpu_work(n):
```

```
    # Pure CPU crunch
```

```
    count = 0
```

```
    for i in range(n):
```

```
        count += i * i
```

```
    return count
```

```
# Run both experiments with total_work = 50_000_000 split across T workers
```

```
# Test T in [1, 2, 4, 8, 16, 32]
```

Deliverables:

1. **The Empirical Curve:** Plot T (workers) on the X-axis vs. Execution Time on the Y-axis for both `threading.Thread` and `multiprocessing.Pool`.
2. **The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
3. **The Architectural Autopsy:**
 - o Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).
 - o Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*?
 - o Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.